

The Reading Tracker Agent is a multi-agent system that lets users manage a personal reading library through natural language. When a user sends a message, it passes through a fixed three-agent sequential pipeline before a response is returned. The pipeline is: Planner Agent, Executor Agent, Reviewer Agent. Each agent has one clearly defined role, its own input, and a defined output format that the next agent depends on.

Planner Agent

The Planner Agent is the first agent in the pipeline. Its sole responsibility is to read the user's raw message and decide which tool should be called and with what arguments. It does not execute any tool, write any data, or format any response. It reasons about intent only.

Role	Intent interpreter and tool selector
Input	Raw user message string (natural language)
Output	A single JSON object specifying the action and arguments
LLM used	Yes — the LLM reads the message against a system prompt listing all available tools and an intent mapping guide
Example input	"How many pages do I have left in Dune?"
Example output	<pre>{"action": "get_pages_left", "args": {"title": "Dune"}, "reasoning": "user is asking for remaining pages"}</pre>

The Planner's system prompt defines all 11 available tools by name, their argument types, and an intent mapping guide that links common phrasings to the correct tool. For example, phrases like "what am I reading" or "current books" map to `get_currently_reading`, while "pages left" or "how

far am I" map to `get_pages_left`. This prevents the Planner from defaulting to a generic tool like `get_library` when a more specific one exists.

Executor Agent

The Executor Agent is the second agent in the pipeline. It receives the Planner's JSON output, looks up the named tool in the tool registry, and runs the corresponding Python function. The Executor does not involve the LLM at any point. It is purely a Python dispatcher.

Role	Tool dispatcher and data layer interface
Input	JSON object from Planner: <code>{"action": "...", "args": {...}}</code>
Output	Python dictionary containing the tool result (status, message, and any book data)
LLM used	No
Example input	<code>{"action": "get_pages_left", "args": {"title": "Dune"}}</code>
Example output	<code>{"status": "success", "title": "Dune", "pages_read": 180, "total_pages": 412, "pages_left": 232, "percent_done": 43.7}</code>

The Executor maintains a registry called `TOOLS` that maps action name strings to Python functions. Tools with no arguments (such as `get_library`, `get_stats`, `get_currently_reading`) are dispatched without unpacking args. Tools that require arguments (such as `add_book`,

update_progress, get_pages_left) receive their arguments via keyword unpacking from the args dictionary.

Reviewer Agent

The Reviewer Agent is the final agent in the pipeline. It receives both the user's original message and the raw tool result from the Executor, then writes a friendly, readable response for the user. It does not modify any data. Its only job is formatting and tone.

Role	Response formatter and communicator
Input	The original user message + the Executor's tool result dictionary (both as context in a single prompt)
Output	A plain-text natural language response shown to the user
LLM used	Yes — the LLM is instructed to be warm, clear, and to present book data in a readable format
Example input	User asked: "how many pages left in Dune?" Tool returned: <code>{pages_left: 232, percent_done: 43.7}</code>
Example output	You have 232 pages left in Dune, which puts you at about 44% done. Keep it up!

The Reviewer's system prompt instructs it to base its response strictly on the tool result provided, not on its own knowledge. It handles both success states (formatting book lists, progress figures,

statistics) and error states (explaining what went wrong in plain language and suggesting a next step).

How the Agents Communicate

The agents do not communicate directly with each other. Communication flows through the `run_agent_pipeline()` function in `app.py`, which acts as the coordinator. This mirrors the `SequentialAgent` pattern from Week 5 of the course, where each agent's output is passed as input to the next step.

The coordination mechanism is a fixed pipeline, meaning the order is always Planner then Executor then Reviewer. There is no dynamic routing, no loop, and no agent choosing which agent to call next. The pipeline is:

- 1) The user message is sent to the Planner via the LLM API. The Planner returns a JSON string.
- 2) The JSON is parsed using regex (the `extract_first_json` pattern from Week 3). The parsed object is passed to the Executor.
- 3) The Executor looks up the tool, runs it, and returns a Python dictionary.
- 4) The dictionary and the original user message are both passed to the Reviewer via the LLM API. The Reviewer returns the final response string.

5) The Flask route returns the response string and the current library state as a JSON payload to the frontend.

No shared memory or state is maintained between pipeline runs. Each user message triggers a completely fresh pipeline execution. The only persistent state is the BOOK_STORE dictionary, which all tools read from and write to directly.

Planner failures

The Planner can fail in two ways: the LLM API call itself can throw an exception, or the model can return a response that does not contain valid JSON. Both are caught by a try/except block in run_agent_pipeline(). If either failure occurs, the pipeline falls back to calling get_library with no arguments, which always succeeds and returns a safe, meaningful response. This means the user always sees something useful rather than a blank error.

If the Planner returns a valid JSON object but specifies an action name that does not exist in the TOOLS registry, the Executor catches this and returns an error dictionary with a descriptive message. The Reviewer then translates this into a plain-language explanation for the user.

Executor failures

Each tool function validates its inputs before touching BOOK_STORE. If a user asks to update progress on a book that is not in their currently reading list, the tool returns a dictionary with

status set to "error" and a message explaining the problem. If a user tries to look up a book that does not exist in any shelf, the tool similarly returns an error dictionary. The Executor does not raise Python exceptions for these cases — it always returns a dictionary, which the Reviewer can handle gracefully.

Reviewer failures

If the LLM API call in the Reviewer step fails, the pipeline catches the exception and falls back to returning the raw message from the tool result dictionary directly. This means the user still receives an accurate response (the tool result), just without the friendly formatting the Reviewer would normally add.

Low-quality Planner output

The Planner may occasionally select a tool that is technically valid but not the most specific match for the user's question. For example, it might call `get_library` when `get_currently_reading` would be more precise. This does not cause an error — the user still receives accurate data. The Planner's intent mapping guide was written specifically to reduce this type of imprecision by explicitly listing common phrasings and their preferred tools. Future improvement would involve adding a Critic agent in a loop pattern (from Week 5) that checks whether the Planner's tool choice matches the user's intent before the Executor runs.

Summary table

Failure type	Handling strategy
Planner API exception	Fall back to get_library, always returns safely
Planner returns invalid JSON	Regex extraction fails, falls back to get_library
Planner picks unknown tool	Executor returns error dict, Reviewer explains to user
Tool cannot find requested book	Tool returns error dict with descriptive message
Reviewer API exception	Falls back to raw tool result message string
Imprecise tool selection	Intent mapping guide reduces frequency; no crash risk